Corso di Sistemi Operativi

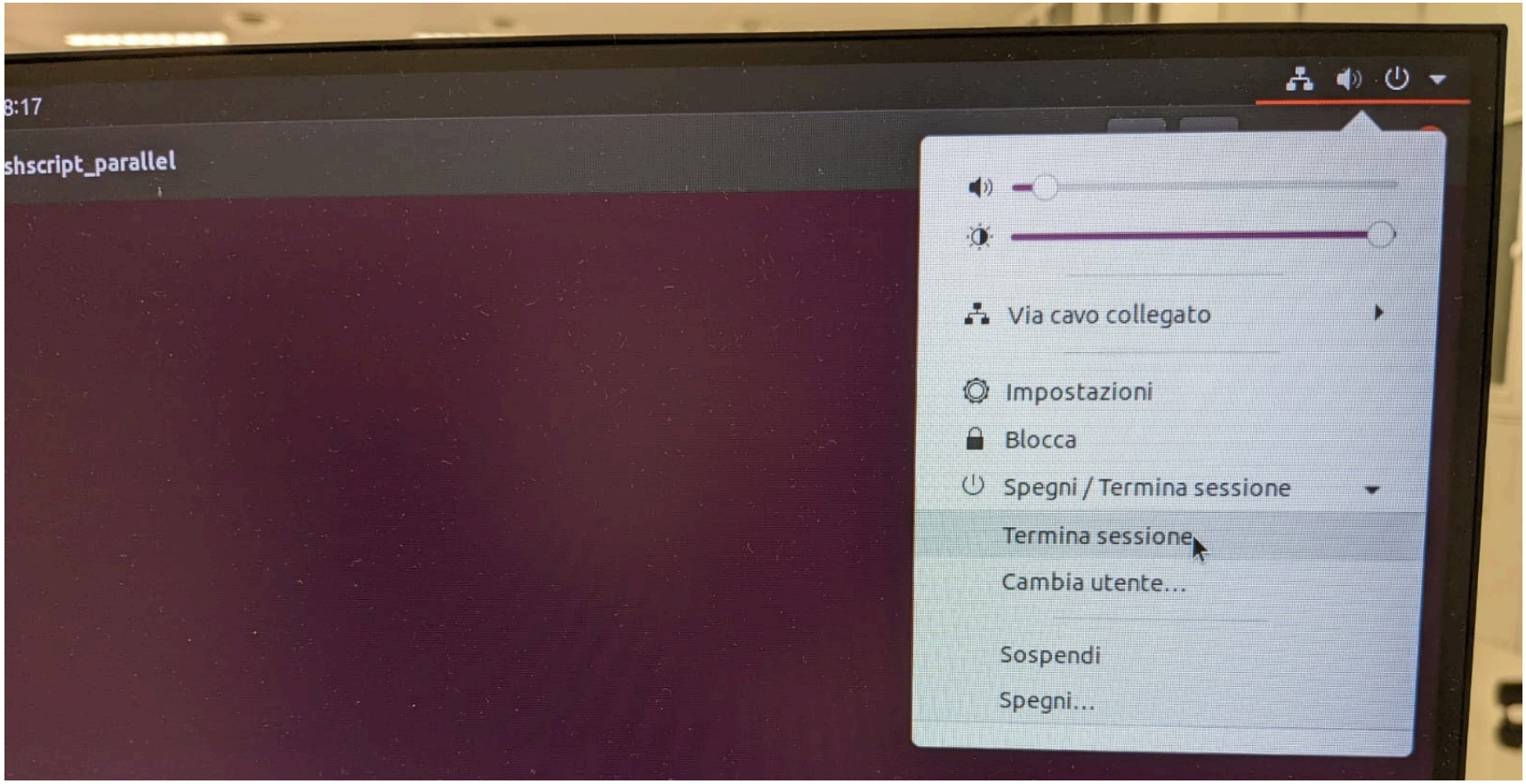
Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Novembre 2025

Appello riservato ai fuori corso

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

Punto 1

Fai in modo che le stampe pre-esistenti vengano visualizzate solo quando attivi una modalità che chiamerai 'debug', mentre l'unica stampa dovrà essere una sorta di tabellone di riepilogo dei risultati attuali che compare ciclicamente, indicando lo stato di ogni partita, le rispettive quote, e quante scommesse attive ci sono su ogni possibile risultato.

Punto 2

Introduci la possibilità per ciascun giocatore di aspettare che si verifichi un certo risultato per una certa partita, con un metodo apposito che chiamerai `attendi_risultato(self,ris)`. Questo metodo va in attesa bloccante fintantoché il risultato attuale della partita in esame non diventa pari a `ris` oppure allorquando la partita termina.

Punto 3

Sfrutta il metodo di cui al punto 2 nell'implementazione del Thread `GiocatoreAffezionato`. Un giocatore affezionato ha una sua squadra preferita, sulla quale scommette sempre, ma solo se tale squadra sta vincendo.

Punto 4

Introduci la possibilità di fare puntate combinate su due partite insieme. I giocatori devono poter scegliere una coppia di partite esistenti, sapere la quota di ogni combinazione (11, 1X, 12, X1, XX, X2, 21, 2X, 22), poter scommettere sulla combinazione scelta, ed essere eventualmente ripagati allorquando **entrambe** le partite terminano.

SALVA SPESSO

ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

NOTA BENE: In questo esercizio dovrai sistemare errori nello script che trovi nella cartella della traccia e non implementarne uno da zero. Leggi la traccia fino in fondo prima di procedere.
CREA TU EVENTUALI FILE DI TESTING.

Lo script `novembre_25.pl` dovrebbe effettuare semplici operazioni di ricerca e comparazione tra files.

```
perl novembre_25.pl [opzioni] [file1 file2]
```

Il parametro `opzioni` dovrebbe poter assumere i valori `-d/-s` mentre `file1` e `file2` sono i **path** a due file di testo.

A. Nel caso in cui l'utente inserisca l'opzione `-d`, lo script dovrebbe comparare riga per riga il contenuto di questi file stampando in output le eventuali differenze come mostrato nell'esempio seguente.

Esempio: Supponendo che i file siano così formati:

FILE1:
Buona fortuna
Accidenti

FILE2:
Buona vacanza
Accidenti
Divertimento

Lo script dovrebbe mostrare il seguente output:

```
Differenze in riga 0
```

```
Buona fortuna
----------------
Buona vacanza

Differenze in riga 2
 <MISSING>
----------------
Divertimento
```

Infatti alla riga `0` il `file1` e il `file2` hanno contenuti completamente differenti mentre la riga 2 non esiste nel `file1` (`<MISSING>`) ma è presente nel `file2`.

B. Nel caso in cui l'utente inserisca l'opzione `-s`, lo script dovrebbe contare il numero di righe del `file2` in cui compare ciascuna parola contenuta in `file1`. Infine si dovrebbe stampare in output la parola trovata (nell'ordine in cui appare in `file1`) seguita dal numero delle sue occorrenze nel `file2` come mostrato nell'esempio seguente.

Esempio: Supponendo che i file siano così formati:

FILE1:

Buona fortuna
Accidenti
Buona

FILE2:

Buona vacanza
Accidenti
Divertimento

Lo script dovrebbe mostrare il seguente output:

```
Buona --> 1
fortuna --> 1
Accidenti --> 0
Buona --> 1
```

Infatti le parole contenute all'interno del `file1` sono:

`Buona, fortuna, Accidenti`

La parola fortuna non compare mai in `file2`, mentre le altre compaiono 1 sola volta.

Lo script allegato, però, presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento sopra descritto. ***NON devi stravolgere il contenuto dello script: la sua struttura deve rimanere pressoché invariata.***